

Diffusion-MoE Phase 1 Investigation — Methodology

Reference

Models, architecture, assumptions, and the diagnostics catalog

Living reference · `diffusion-moe project` · kept in sync with `reports/phase1_findings_report.md`

1. Model Under Test

All diagnostics in this investigation run against the **pretrained dense reference model**, not the project's own (not-yet-trained) `DiffusionMoETransformer`. This is a deliberate distinction worth keeping explicit: the goal is to validate the manifold/routing/width-reduction hypotheses *before* training anything, using an existing model's real activations as a stand-in for what a trained diffusion-MoE model's dense backbone would look like.

1.1 `mistralai/Mistral-7B-v0.1`

Pulled directly from the model's Hub config (`AutoConfig.from_pretrained`), not assumed:

Spec	Value
Architecture	<code>MistralForCausalLM</code> (decoder-only, RoPE, GQA, SwiGLU MLP, RMSNorm, pre-norm)
Layers (decoder blocks)	32 — indices 0–31
Hidden size (d_model)	4096
FFN intermediate size (d_ff)	14336 (SwiGLU, ~3.5× d_model — not 4× d_model as the project's own toy 300M/1.3B configs assume)
Attention heads	32 query heads, 8 KV heads (grouped-query attention, 4:1 ratio)
Head dim	128
Vocab size	32000
Max position embeddings	32768
Sliding window (local attention)	4096 tokens
RoPE theta	10000
Norm	RMSNorm, eps = 1e-5
Activation	SiLU (SwiGLU gate)
BOS / EOS token ids	1 / 2
Checkpoint dtype	bfloat16

Layer-index convention used throughout: layer `i` refers to `model.model.layers[i]`. "Post-attention" for layer `i` means the raw output of `model.model.layers[i].self_attn` (captured via a forward hook, *before* the residual add and before the FFN) — this is the point `DiffusionMoELayer` is specified to route from (see `moe_layer.py` 's design), not the block's final output. "Pre-attention" for layer `i` is HF's `output_hidden_states[i]` (the residual stream entering block `i`; for `i=0` this is the raw token embedding).

Not used, to avoid confusion: `configs/model/300m.yaml` and `configs/model/1.3b.yaml` describe the project's *own* target architectures for eventual training (Phase 2) and share no numbers with `Mistral-7B-v0.1` above. Nothing in this investigation has run those configs.

2. Key Assumptions

Recorded here so they're easy to challenge or revisit, not because they're all necessarily right.

Assumption	Where it matters	Status
Model exposes <code>model.model.layers[i].self_attn</code> and <code>.mlp.down_proj</code> (Llama/Mistral module layout)	Every hook-based capture (<code>AttentionOutputCapture</code> , <code>MLPInputCapture</code>) raises <code>ValueError</code> on models that don't match this	Holds for Mistral-7B-v0.1; would need adaptation for other architectures
"Post-attention" = raw <code>self_attn</code> output, pre-residual-add	All geometry/routing diagnostics	Matches the project's own routing-point spec; not independently re-derived
Compute dtype: bf16 on GPU/MPS, fp32 on CPU; diffusion-map math upcasts to float64 internally regardless of model dtype	All scripts	Chosen for speed; bf16 rounding was an early candidate explanation for kernel disconnection — investigated and ruled out (<code>duplicate_fraction</code> stayed flat ~0.5%, not elevated)
BOS token gets added automatically by the tokenizer's default call	Original hypothesis about position-0 attention sinks	Revised by evidence — position 0 shows zero representation among norm outliers (§4 of the findings report); the actual outlier-carrying token is the newline character, not position 0
<code>max_seq_len</code> (512 in most scripts, 256 in <code>sink_token_diagnostic.py</code>) stays well inside Mistral's 4096 sliding window	All scripts	True by construction — no windowing-truncation confound in any result so far
Diffusion-map hyperparameters (<code>n_landmarks=128</code> , <code>n_components=32</code> , <code>diffusion_t=3</code> , <code>alpha=1.0</code> , <code>intrinsic_dim_threshold=0.95</code>)	All geometry scripts	Inherited from the project guide's Section 5/9 spec, not independently tuned or re-derived by this investigation
Kill-switch " <code><</code> " threshold: <code>r* < d_model / 10</code>	<code>extract_geometry.py</code> 's pass/fail message	A judgment call made when fixing the original <code>d_model × 0.5</code> bug — an order-of-magnitude convention, not a value specified anywhere in the guide
Disconnection heuristic: likely_disconnected fires when the leading non-trivial eigenvalue > 0.999	<code>extract_geometry.py</code> , <code>multiscale_geometry.py</code>	An empirically chosen cutoff (comfortably below the theoretical trivial eigenvalue of exactly 1.0), not derived from a formal test
wikipedia (<code>wikimedia/wikipedia</code> , <code>20231101.en</code>) is the primary dataset per the project guide; wikitext (<code>Salesforce/wikitext</code> , <code>wikitext-103-raw-v1</code>) is used as a fallback when the former's CDN is flaky	Every script defaults to wikipedia; several runs in this investigation used — dataset wikitext	Not interchangeable for exact replication — a wikitext-based run is not a byte-for-byte comparison to a wikipedia-based run at the same layer. The layer-31 punctuation/units finding (§4.2 of the findings report) is suspected to be partly a wikitext-103-specific numeric-escaping artifact (<code>@,@,@.@</code>) for exactly this reason

Random seed: 42, fixed across all scripts	Landmark selection (k-means), token subsampling, everything downstream	Consistent within a run; cross-run comparability still depends on same dataset + same n_sequences (not always held constant across this investigation's runs — see findings report caveats)
Plain, unweighted k-means on raw diffusion coordinates is an adequate stand-in for DiffusionRouter's actual (load-balanced) token-to-expert assignment	ffn_specialization.py's clustering step	Revised by evidence — raw-Euclidean k-means collapsed roughly 99% of tokens into one cluster at every layer tested (the real router's <code>coefficient_of_variation_loss</code> exists specifically to prevent this collapse, and wasn't replicated here). Fixed by clustering on cosine-normalized coordinates instead, which restored balanced (roughly 45–51% max-cluster-share) partitions at most layers tested — see findings report §3.1
<code>configs/base_config.yaml</code> 's <code>tau: 0.1</code> is a reasonable routing temperature regardless of layer/model	DiffusionRouter's dispatch softmax, <code>total_loss</code> 's load-balance term	Revised by evidence, seriously — real diffusion coordinates measured $\sim 1e-7$ in magnitude (small eigenvalues raised to <code>diffusion_t=3</code>), making <code>tau=0.1</code> produce a dispatch softmax numerically indistinguishable from uniform on real Mistral-7B data, confirmed directly. <code>tau</code> implicitly assumed $O(1)$ -scale logits; fixed by rescaling coordinates by <code>landmark_scale</code> before applying <code>tau</code> — see findings report §3.4
ARPACK's iterative eigensolver (<code>scipy.sparse.linalg.eigs</code>) reliably converges for any kernel bandwidth	<code>diffusion_eigenvectors</code> , used by every diagnostic's <code>NystromDiffusionMap.fit</code>	Revised by evidence — hit a real <code>ArpackNoConvergence</code> at layer 25 during the full 32-layer multiscale sweep, at one of the extreme dyadic-ladder bandwidths where the kernel's eigenspectrum becomes near-degenerate. Fixed with a dense-solver fallback on that exception — see findings report §2.4

3. Diagnostics Catalog

Every diagnostic below is a real CLI script under `scripts/`, backed by tested, reusable logic under `src/diffusion_moe/geometry/`. All write JSON results (and most a PNG plot) to `results/geometry/`.

3.1 `scripts/extract_geometry.py` — Phase 1 kill switch

Purpose: per-layer intrinsic dimension (r^*) and spectral gap, to decide whether the manifold hypothesis holds before committing to Phase 2 training.

Measures per layer: `r_star_post_attention`, `r_star_pre_attention`, `spectral_gap`, `top_eigenvalues`, Nyström approximation error vs. landmark count, `likely_disconnected` flag, and `pool_diagnostics` (`eps` — the fitted kernel bandwidth; `token_norm_max_to_median`; `duplicate_fraction`; `n_tokens`).

Key defaults: `n_landmarks=128`, `n_components=32`, `diffusion_t=3`, `alpha=1.0`, `intrinsic_dim_threshold=0.95`, `max_pool_size=8192` tokens/layer, `n_sequences=1000`,

`max_seq_len=512` , `batch_size=8` , `seed=42` , dataset default `wikipedia` (now overridable via `--dataset / --local_data_files` , added for the same CDN-reliability reason as `StreamingTextDataset` 's `local_data_files` — see §4's infrastructure notes).

Also supports `--exclude_token_ids` (default: none, so behavior is unchanged unless requested): drops matching tokens from geometry pooling entirely, via `activation_capture.py::collect_layer_activations` 's new `excluded_token_ids` parameter — see findings report §2.5.

Output: `results/geometry/{model}_geometry.json` , `{model}_geometry.png` .

```
python scripts/extract_geometry.py --model mistralai/Mistral-7B-v0.1 --n_sequences 1000
```

Status: rerun at full defaults after the threshold/diagnostics fix, reproducing the original 4 disconnected layers and r^* statistics (mean 4.6, median 4.5, range 1–11) — confirms the earlier partial-sample result wasn't a sampling artifact. See findings report §2.3. A separate 200-sequence/wikitext run comparing `--exclude_token_ids 13` (newline) against a baseline found the exclusion cleared both layers flagged disconnected in that run — findings report §2.5.

3.2 `scripts/multiscale_geometry.py` — dyadic bandwidth sweep

Purpose: tests whether a layer's disconnection/ r^* is a single-bandwidth artifact or persists across scale, in both raw-Euclidean and cosine/angular metrics, and whether a stable r^* plateau exists in either.

Measures per layer, per metric: a 9-point dyadic ladder (`eps_center × 2k` , `k ∈ [-4, 4]`) of {`eps` , `eps_ratio_to_median` , `r_star` , `spectral_gap` , `top_eigenvalues` , `likely_disconnected`} , plus the longest stable (non-disconnected, constant- r^*) window found.

Key defaults: same diffusion-map hyperparameters as §3.1; `n_scales=9` , `base=2.0` , `n_sequences=200` , `max_seq_len=512` , dataset default `wikipedia` .

Output: `results/geometry/{model}_multiscale_layer{N}.png` , `{model}_multiscale.json` .

```
python scripts/multiscale_geometry.py --model mistralai/Mistral-7B-v0.1 --dataset wikitext --n_sequences 200 --layers 1 2 4 15 31
```

Status: run at full 32-layer coverage (`results/multiscale_full32/`). Only layers 17, 20, and 31 never connect under raw Euclidean at any of the 9 bandwidths tested — layers 1/2/4's disconnection turned out to be specific to the kill switch's single default bandwidth, not present across the sweep. Where both metrics connect (29 layers), they agree exactly in 10, Euclidean reports higher r^* in 13, cosine higher in 6. See findings report §2.4 for the full reading and its implication for the production-kernel decision.

A real bug hit and fixed while running this at full coverage: `diffusion_eigenvectors` 's ARPACK-based eigensolver (`scipy.sparse.linalg.eigs`) raised an uncaught `ArpackNoConvergence` at layer 25, one of the extreme bandwidths in the sweep — plausible, since those bandwidths are exactly where the kernel matrix's eigenspectrum can become near-degenerate, which is what makes an iterative solver struggle to converge in the first place. Fixed by falling back to a dense solver (`scipy.linalg.eig`) on that exception, same as the existing fallback for matrices too small for ARPACK's own constraints — see §2's assumptions table.

3.3 `scripts/ffn_specialization.py` — width-reduction hypothesis test

Purpose: tests whether narrow per-expert FFNs (the G2 compute-savings claim) are justified, independent of whatever r^* says about routing — by measuring, per candidate expert count K , how much of a diffusion-cluster's real FFN neuron-activation mass a width- `d_ff/K` slice of its own top neurons captures vs. a cluster-agnostic globally-shared slice of the same size.

Measures per layer, per K in `n_experts` : for BOTH a diffusion-based clustering and an oracle clustering (see below), `mean_specialization_gain` (own-cluster coverage minus shared-slice coverage — large & positive supports narrow experts), `mean_pairwise_jaccard` (top-neuron-set overlap between clusters — near 1 argues

against specialization), per-cluster detail; plus `agreement` (Adjusted Rand Index, Normalized Mutual Info) between the two clusterings.

Why two clusterings, not one: a null result against diffusion clustering alone is ambiguous — the dense model was never trained with any incentive to organize around diffusion-cluster boundaries, so "no specialization found" could mean either "not achievable" or "achievable, but this particular untrained routing signal doesn't find it" (a real MoE's load-balancing loss would reshape neuron usage during training in a way nothing training-free can simulate). The **oracle** clustering — k-means directly on the FFN activations themselves (PCA-reduced to the same dimensionality as the diffusion coordinates, for a fair comparison) — sidesteps this: it's the best-case K-way partition for this exact metric, by construction, giving a training-independent ceiling on achievable specialization. Comparing diffusion-clustering's result against that ceiling (via the agreement metrics) separates "is specialization achievable at all" from "does diffusion geometry find it."

Key defaults: same diffusion-map hyperparameters as §3.1; `n_experts=[4, 8, 16]` (matches `configs/base_config.yaml`'s own sweep grid), `n_sequences=200`, `max_seq_len=512`, dataset default `wikipedia`. The diffusion clustering runs on **cosine-normalized** diffusion coordinates (via `geometry.multiscale.l2_normalize`), not raw ones — see the assumption/fix noted in §2 and the findings report §3.1: raw-Euclidean clustering was found to collapse ~99% of tokens into one k-means cluster (the same outlier-norm tokens responsible for the kill-switch disconnection dominate plain k-means too), which cosine normalization fixes without needing to know which token identity is responsible. `mlp_activations` — what specialization is actually measured against, for both clusterings — stays raw/unnormlized regardless; only the clustering inputs change.

Output: `results/geometry/{model}_ffn_specialization_layer{N}.png` (oracle vs. diffusion gain, plus ARI, per K), `{model}_ffn_specialization.json`.

Status: run at full 32-layer coverage (`results/ffn_full32/`, `wikitext`, 200 sequences), after an initial 5-layer pass. The full-coverage result is more lopsided than the 5-layer sample suggested: 23 of 32 layers show a real, currently-unexploited oracle ceiling; only 3 (layers 10, 29, 30) show diffusion clustering recovering a meaningful part of it ($\text{ARI} \geq 0.2$); only 6 layers show no achievable specialization at all. Layer 3 (untested in the 5-layer pass) has the single highest oracle ceiling of any layer. See findings report §3.3.

```
python scripts/ffn_specialization.py --model mistralai/Mistral-7B-v0.1 --dataset wikitext --layers 1 2 4 31 15 --n_experts 4 8 16
```

3.4 `scripts/sink_token_diagnostic.py` — outlier token identity

Purpose: identifies *which* tokens (by sequence position and by token identity) drive the norm-outlier disconnection found in §3.1/§3.2, to distinguish a fixed-position sink story from a token-identity-anchored one, and to test whether elevated norm is specific to one token or shared by a broader delimiter/punctuation class.

Measures per layer: position-based (`frac_at_position_0`, `frac_at_last_position`, `median_dist_from_start/end`, decoded text of top- `top_frac` outlier instances) and identity-based (`per_token_id_norm_summary` — mean/median/max norm per unique token id with $\geq \text{min_token_id_count}$ occurrences; `delimiter_vs_content` — mean norm compared between delimiter-like and content token-id groups, via a tokenizer-agnostic "contains no alphanumeric characters" classifier).

Key defaults: `top_frac=0.01`, `min_token_id_count=3`, `n_sequences=50`, `max_seq_len=256`, `layers=[1, 2, 4, 15, 31]`, dataset default `wikipedia` (also supports `--local_data_files`, same reason as §3.1).

Output: `results/geometry/{model}_sink_token_diagnostic.json`.

```
python scripts/sink_token_diagnostic.py --model mistralai/Mistral-7B-v0.1 --dataset wikitext --n_sequences 50 --layers 1 2 4 15 31
```

Status: also run against `the_pile` (`--dataset the_pile`, same layers) to test whether layer 31's wikitext-specific punctuation pattern (§4.2 of the findings report) generalizes — it doesn't (delimiter-class ratio 2.2x on wikitext

vs. 1.15x on `the_pile`), while newline-token dominance does (still the single largest outlier by a wide margin on both). See findings report §4.4.

3.5 Underlying reusable library (`src/diffusion_moe/geometry/`)

Module	Provides
<code>kernel.py</code> , <code>markov.py</code> , <code>eigensolver.py</code> , <code>intrinsic_dim.py</code>	Core diffusion-map math (Gaussian kernel, Coifman-Lafon normalization, eigendecomposition, r^* /spectral-gap estimation) — pre-existing project code
<code>nystrom.py</code>	<code>NystromDiffusionMap</code> — landmark-based Nyström approximation; extended in this investigation with an optional <code>eps</code> override (enables the §3.2 bandwidth sweep while holding landmarks fixed)
<code>activation_capture.py</code>	<code>AttentionOutputCapture</code> (post-attention hook), pooling/subsampling helpers shared by §3.1–§3.3
<code>multiscale.py</code>	<code>l2_normalize</code> , <code>dyadic_eps_ladder</code> , <code>multiscale_diffusion_analysis</code> , <code>find_stable_window</code> — backs §3.2
<code>ffn_specialization.py</code>	<code>MLPInputCapture</code> (pre-hook on <code>mlp.down_proj</code>), <code>cluster_tokens</code> , <code>neuron_specialization_analysis</code> , <code>oracle_cluster_tokens_by_activation</code> , <code>cluster_agreement</code> — backs §3.3
<code>sink_diagnostics.py</code>	<code>token_norms_from_capture</code> , <code>summarize_outlier_positions</code> , <code>per_token_id_norm_summary</code> , <code>is_delimiter_like</code> , <code>compare_delimiter_vs_content_norms</code> — backs §3.4

All of the above are covered by unit tests against synthetic data with known ground truth (e.g. a Swiss roll with known $r^*=2$, synthetic disjoint-vs-uniform neuron usage, synthetic start-anchored vs. end-anchored outlier positions) — see `tests/geometry/` and `tests/scripts/` for the corresponding real-model-independent test suites.

4. Bounded Pilot Fine-Tune (First Training-Based Evidence)

Every diagnostic in §3 is training-free — none of them can distinguish "specialization isn't achievable at this layer" from "achievable, but not found by an untrained, fixed routing signal" (§3.3's oracle-ceiling result narrowed this, but didn't eliminate it — a trained MoE's load-balancing loss actively reshapes neuron usage in a way nothing training-free simulates). This section is the first departure from that: an actual, deliberately small and cheap training run, to get real (not proxy) evidence at the layers §3.3 flagged as having a genuine oracle ceiling.

4.1 Compute: vast.ai instance

Local Apple Silicon (MPS) has no DDP backend and untested backward-pass support for this project's custom scatter/gather MoE dispatch, so training moved to a rented CUDA box — the same infrastructure the project's own `Makefile` 's `train-distributed` target and `device.py` 's `setup_distributed` already assume for Phase 2, just a single GPU rather than a DDP cluster (a single 24GB Ampere+ card is real overkill-avoidance for this scope; see the recommendation-selection discussion this investigation went through before renting).

Spec	Value
Provider	vast.ai (API key in <code>.env</code> as <code>VAST_API_KEY</code>)
GPU	1x RTX 3090, 24GB VRAM, Ampere (compute capability 8.6 — confirmed via <code>torch.cuda.get_device_capability()</code> , needed for native bf16 tensor-core support)
Instance	contract id 50914670, offer id 49828263, labeled <code>diffusion-moe-pilot</code>
Image	<code>pytorch/pytorch:2.4.0-cuda12.4-cudnn9-devel</code>

Cost	~\$0.219/hr total (GPU \$0.147/hr + 60GB disk \$0.072/hr)
Access	ssh -p 34670 root@ssh6.vast.ai (key already registered on the vast.ai account)

Dependency pinning gotcha worth recording: the image ships PyTorch 2.4.0. A plain `pip install transformers` grabs the latest release (5.17.0 at the time), which requires PyTorch ≥ 2.5 and *silently* disables its PyTorch backend rather than erroring loudly ("PyTorch was not found. Models won't be available"). Pinned to `transformers==4.44.2` instead — compatible with 2.4.0, confirmed by importing `AutoModelForCausalLM` and loading a real model. `requirements.txt` 's own `torch==2.3.0+cu121` pin was NOT installed on this box (would have fought the image's working, GPU-verified 2.4.0 install for no benefit) — installed everything else it lists instead (`pip install --no-deps -e .` for the package itself, plus the rest of the dependency list individually).

Deploying code: `rsync` from the local working tree to `/root/diffusion-moe` on the instance (not a git clone/push — sidesteps needing to commit work-in-progress just to move it). One real bug hit here: an early `--exclude='data'` pattern is unanchored and matched `src/diffusion_moe/data/` (the actual Python package) as well as the intended top-level cache directory — `--exclude='/data/'` (anchored to repo root) is the correct form. Full test suite (307 tests) re-run and passing on the remote box after sync, confirming the environment, not just the code, is equivalent.

4.2 src/diffusion_moe/models/pilot_moe_block.py — PilotMoEBlock

Unlike `models/moe_layer.py` 's `DiffusionMoELayer` (which owns its own attention and replaces a WHOLE transformer block in the project's from-scratch architecture), `PilotMoEBlock` replaces ONLY a pretrained decoder layer's `.mlp` submodule — attention and every other layer stay frozen and pretrained. Its `forward(x) -> Tensor` signature matches what HF's `MistralDecoderLayer.mlp` expects, so splicing in is exactly `layer.mlp = PilotMoEBlock(...)`.

Reuses the project's existing routing components directly, unmodified: `NystromDiffusionMap` (cosine-normalized by default, consistent with §2's findings), `ExpertCentroids`, `DiffusionRouter`, `ExpertFFN`, `dispatch_and_aggregate`. Since the surrounding frozen decoder layer only expects a plain tensor back (not the (output, aux) tuple `DiffusionMoELayer` returns), auxiliary routing info needed by `training.losses.total_loss` (router logits, centroids, Nyström landmarks) is stashed on `self.last_aux` after each forward call instead, read out by the training script as `{layer_idx: block.last_aux}`.

Trained in fp32 via a thin `DtypeCastWrapper` even though the frozen backbone runs in bf16 — casts the block's input to fp32 and its output back to the backbone's dtype at the boundary, a standard adapter-training pattern for numerical stability of a small trainable module bolted onto a larger frozen one.

Tested with 6 unit tests mirroring `test_moe_layer.py` 's existing coverage (output shape, centroid initialization, step-counter/refresh-schedule behavior, gradient flow, and — the one genuinely new check — that `last_aux` plugs directly into `total_loss` the same way any of the project's own MoE layer variants' aux dicts do).

4.3 scripts/pilot_finetune.py

Freezes the entire pretrained model, splices in one `PilotMoEBlock` at `--layer` (default 2 — the strongest oracle-ceiling candidate from §3.3), enables gradient checkpointing (`model.gradient_checkpointing_enable()` + `model.enable_input_require_grads()` — the latter is required specifically because checkpointing needs a tensor with `requires_grad=True` flowing through, which a fully-frozen embedding otherwise breaks), and trains only the new block's parameters (176M of Mistral-7B's 7.24B, ~2.4%, confirmed by direct count in the first real run) with AdamW + linear-warmup-then-cosine schedule, reusing `training/optimizer.py` and `training/losses.py` unmodified.

Success signal: does `task_loss` fall substantially from its step-0 value (and ideally approach a **dense baseline** — one forward pass through the model AS-PRETRAINED, before the splice, on a held-out first batch — captured before the MLP is replaced) while `load_loss` stays low (routing not collapsing onto one expert)? That's direct evidence the architecture can learn to specialize here, as distinct from whether it already happens to for free (§3.3's question).

Key defaults: `--n_experts 8 --top_k 2` (matches `configs/base_config.yaml`), `--mu 0.01 --nu 0.05` (load-balance / separation loss weights, also matching the base config), `--centroid_refresh_steps 20` (much

shorter than `DiffusionMoELayer` 's production default of 500 — a bounded pilot has far fewer total steps to begin with), `--steps 300` , `--batch_size 4` , `--max_seq_len 256` , dataset default `wikipedia` (real runs used `--dataset wikitext` for the same CDN-reliability reason as the other scripts). Logs to Weights & Biases automatically when `WANDB_API_KEY` is set in `.env` (project `diffusion-moe`) — confirmed working end-to-end in the smoke test (an early run briefly logged to a project named `diffusion-moe-pilot` before this was consolidated into the single `diffusion-moe` project).

Status: five full 300-step runs total — the first two surfaced real routing bugs (§4.4) rather than clean results; the third (post-fix, default `mu=0.01` / `centroid_refresh_steps=20`) completed with `task_loss` closing roughly 42% of the gap to the dense baseline (best point roughly 62%, at step 270) alongside a new finding — `load_loss` oscillates between full collapse (`7.0` , `coefficient_of_variation_loss` 's max for `n_experts=8`) and good balance (roughly `0.0001`) across the run. Two further tuning runs (§4.5) tested whether stopping periodic landmark re-fitting and/or raising `mu` resolves the oscillation; neither did conclusively. Each run is smoke-tested at 5-8 steps first (confirmed correct trainable-param count, finite losses, working W&B logging, no crashes) before committing to the full run.

```
python scripts/pilot_finetune.py \
  --model mistralai/Mistral-7B-v0.1 --dataset wikitext \
  --layer 2 --n_experts 8 --top_k 2 --steps 300 \
  --batch_size 4 --max_seq_len 256
```

4.4 Two bugs in shared routing code, found only by training

Both bugs live in `src/diffusion_moe/routing/` — shared code the production `DiffusionMoELayer` also uses, not anything pilot-specific — and both are dynamics/scale issues no training-free diagnostic in this investigation could have caught. Full narrative in the findings report §3.4; summarized here for the assumptions/tooling record.

Bug	Symptom	Fix
Unbounded centroid growth	<code>load_loss</code> exactly <code>0.0000</code> for all 300 steps; <code>sep_loss</code> grew from <code>-3</code> to <code>-19,185</code>	<code>ExpertCentroids.clip_norm()</code> — caps centroid norm at a multiple of <code>landmark_scale</code> after each optimizer step
Degenerate router (deeper, universal)	<code>load_loss</code> <i>still</i> exactly <code>0.0000</code> after the fix above; on real layer-2 activations, tempered (<code>tau=0.1</code>) dispatch read exactly <code>[0.125]×8</code> for every token	Route on <code>Ψ_t/centroids</code> rescaled by <code>landmark_scale</code> before the tempered softmax, in both <code>PilotMoEBlock</code> and <code>DiffusionMoELayer</code>

Root causes: bug 1 — `centroid_separation_loss` 's gradient has no upper bound on centroid separation (intentional — see `test_more_spread_centroids_give_more_negative_loss`), so nothing stops centroids drifting outside the data's real coordinate range, at which point every token becomes roughly equidistant from every centroid. Bug 2 — real `Ψ_t` magnitude measured `~1e-7` (small eigenvalues raised to `diffusion_t=3`), far too small for an `O(1)`-scale `tau=0.1` to produce anything but a numerically-uniform softmax regardless of which centroid is actually closest.

Second-order bug found while fixing the second one: `landmark_scale` 's zero-guard epsilon (originally `1e-8`) was itself large enough to dominate and silently cap the rescaling when the true coordinate scale was smaller still (down to `~1e-16` in one stress test) — tightened to `1e-30` .

Assumption this revises: `configs/base_config.yaml` 's `tau: 0.1` implicitly assumed router logits are `O(1)` scale. They aren't — real diffusion coordinates are many orders of magnitude smaller, layer- and model-dependent. Anywhere `tau` is used against raw (non-rescaled) diffusion coordinates going forward should be treated as suspect.

Done — see §4.7: both fixes confirmed holding through the production `Trainer / DiffusionMoETransformer` path, not just the pilot's frozen-backbone splice — findings report recommendation 4, §3.6.

4.5 Tuning attempts against the load-balance oscillation

Two follow-up runs, both at `--layer 2` , holding everything else at §4.3's defaults:

Run	Change from §4.3's run	Result
A	<code>--centroid_refresh_steps 10000</code> (effectively fit-once, vs. default 20)	<code>sep_loss</code> fully stabilized (roughly -3.9 to -4.35 throughout) — confirms periodic re-fitting was rotating the coordinate frame under the router and contributing to instability. <code>load_loss</code> got worse, not better: sustained near-collapse (6.4–6.9) in the last third of training
B	Run A's settings, plus <code>--mu 0.3</code> (vs. default 0.01)	Final <code>task_loss</code> worse than either prior run (roughly 25–46% gap closed vs. roughly 42% for both others). <code>load_loss</code> still spikes to 6.3–6.7 mid-run, though it recovers by the end rather than staying collapsed

Run B's `task_loss` trajectory was bit-identical to run A's through step 80 despite the 6x `mu` difference — the load-balance gradient is small enough that even a large reweighting takes many steps to produce visible divergence, which limits how much can be concluded from a single 300-step run at this batch size. Not pursued further this pass; see findings report §3.4/§7 for the assessment and the untried next lever (larger effective batch size for a less noisy load estimate).

4.6 Resolving the instability: batch size, seed-dependence, and noisy top-k gating

Full narrative and results table in findings report §3.5; methodology/tooling notes here.

`--batch_size 16` (vs. the default 4), all else unchanged, eliminated full router collapse over 300 steps on its own and substantially improved `task_loss` (gap closed rose from roughly 42% to roughly 86%).

Per-step expert-dominance logging was added to `pilot_finetune.py` 's training loop (`per_expert_load` , `argmax_expert` in each history record) — the dense per-expert softmax mean and its argmax, alongside the existing aggregate `load_loss` scalar. This is what surfaced that one pair of experts won 83% of all steps' dispatch under the default seed; rerunning with `--seed 123` (identical otherwise) reproduced comparably severe concentration onto a *different* pair, isolating k-means++ initialization luck as the cause rather than a structural property of the layer's geometry.

Noisy top-k gating (`src/diffusion_moe/routing/router.py::DiffusionRouter` , new `noise_std` parameter, default `0.0`): i.i.d. Gaussian noise added to `router_logits` during training only (`self.training gated`), before the tempered softmax — both the top-k dispatch and the aux `router_logits` used by `total_loss` 's load-balance loss see the same noisy signal, so the loss reflects what dispatch actually did. Wired through `PilotMoEBlock` and the production `DiffusionMoELayer` identically. A controlled synthetic test (`tests/routing/test_router.py`) confirms the mechanism works in isolation: a deliberately biased centroid layout (one centroid planted at the token distribution's mean, matching the real "early leader" pattern) shows measurably reduced dispatch concentration under noise, averaged over several seeds.

Calibration pitfall, confirmed on a real run: noise is added *before* the `tau` division, so its effective magnitude in the softmax is `noise_std / tau` . The first real attempt used `--noise_std 1.0` with the default `tau=0.1` — an effective magnitude of roughly 10, large enough to dominate the real routing signal rather than gently perturb it. Result: worse, not better (full-collapse steps rose from 0 to 51 of 300; the dominant expert's share rose from 83% to 95%). Dropping to `--noise_std 0.1` (effective magnitude roughly 1, matching the order of magnitude validated in the synthetic test at `tau=0.5 / noise_std=1.0` , effective magnitude roughly 2) worked as intended: 0 full-collapse steps, mean `load_loss` slightly better than the noise-free run, and meaningfully more experts sharing real dispatch (5 vs. 2), at a small cost to task-loss gap closure (82% vs. 86%).

Infrastructure note — a repeat CDN-flakiness problem, and the actual fix: getting these runs to execute at all required resolving a real, recurring reliability issue. HuggingFace Hub's *streaming* dataset reader (`datasets.load_dataset(..., streaming=True)`) , used throughout this project via `StreamingTextDataset`) stalled indefinitely three separate times fetching wikitext — traced with debug logging to HF's newer "Xet" CDN

backend, which redirects file reads through many small, separately-connected byte-range requests rather than one bulk transfer. Pre-fetching the files with `huggingface_hub.hf_hub_download` (a more robust, resumable downloader) and even rsync-ing the resulting local cache to the training box did **not** fix this — confirmed directly via debug logging that streaming reads go through `fsspec's HffFileSystem`, a different code path that re-fetches over the network regardless of what's already in the local hub cache. The fix that actually worked: a new `local_data_files` parameter on `StreamingTextDataset` (`src/diffusion_moe/data/dataset.py`) that, when given local parquet file paths, calls `load_dataset("parquet", data_files=..., streaming=True)` instead of the registry's Hub repo id — bypassing network reads entirely. Exposed as `--local_data_files` on `pilot_finetune.py`. Tested both for call-argument wiring and end-to-end against a real local parquet fixture (`tests/data/test_dataset.py`).

Separately, a real repo-integrity bug was found and fixed while investigating this: `.gitignore's data/` pattern was unanchored, matching not just the intended top-level `DATA_DIR` cache target but also `src/diffusion_moe/data/` (the actual Python package: `dataset.py`, `tokenizer.py`, `dataloader.py`) and `tests/data/` — meaning this whole package had never been tracked by git, since the initial commit. Fixed by anchoring to `/data/`; the same bug class as an earlier `rsync --exclude='data'` mistake (§4.1), same fix.

4.7 Verified in the actual production training path — and three more bugs found

Full narrative in findings report §3.6. Every result above this point was exercised only through `PilotMoEBlock's` splice into a frozen pretrained model, never through the project's real training entrypoint. Closing that gap (findings report recommendation 4) meant running `scripts/train.py` directly:

```
WANDB_API_KEY= python scripts/train.py \
    model.d_model=128 model.n_heads=2 model.n_layers=2 model.ffn_dim=256 model.max_seq_len=64 \
    routing.n_experts=4 routing.top_k=2 routing.n_components=8 routing.n_landmarks=16 \
    routing.centroid_refresh_steps=5 \
    data.dataset=wikitext data.batch_size=2 data.max_seq_len=64 data.num_workers=0 \
    data.val_tokens=256 \
    training.total_tokens=1024 training.grad_accum_steps=1 training.warmup_steps=0 \
    training.checkpoint_steps=1000000 training.eval_steps=1000000 training.log_steps=1
```

A tiny model (for speed, not to avoid anything) at real defaults otherwise — critically, `training.precision: bf16`, the project's own actual default, not `fp32`.

It crashed immediately, on `TypeError: Got unsupported ScalarType BFloat16 inside DiffusionMoELayer._compute_diffusion_coords's .cpu().numpy()` call, then again at the identical line in `ExpertCentroids.initialise_from_batch`, then again inside `landmark_scale's torch.cdist` call. Root cause in all three: NumPy has no `bfloat16` dtype, and `torch.cdist` has no `bfloat16` implementation — both are hard crashes, not accuracy concerns. `PilotMoEBlock` never hit this because its own `DtypeCastWrapper` already forces `fp32` at the block boundary, for an unrelated reason (training stability of a small module bolted onto a frozen `bf16` backbone) — which is exactly why the pilot's training-based validation couldn't have caught a bug specific to the production path's actual dtype handling.

Fixed: `.float()` before `.numpy()` in `moe_layer.py` and `centroids.py`; `.float()` before `torch.cdist` in `separation.py` (both call sites — `landmark_scale` and `centroid_separation_loss`). All differentiable, no effect on gradient flow. Confirmed with a direct `bf16` forward+backward test (`tests/models/test_moe_layer.py::test_forward_and_backward_work_under_real_bf16_mixed_precision`) and by rerunning the command above to completion: 8 clean steps, `load_loss / sep_loss` finite and non-degenerate throughout.

A fourth gap, unrelated to dtype: `ExpertCentroids.clip_norm()` (§4.4, bug 1's fix) was never called anywhere in `Trainer` — the pilot script's own training loop was its only caller in the entire codebase. Fixed by calling it from `Trainer.train_step()` after every optimizer step, for every block exposing both `centroids` and `ndm` attributes (`getattr` -guarded, so dense `TransformerBlock's` and the non-diffusion router variants are unaffected).

`centroid_max_radius_factor` is now a `Trainer` config field (`routing.centroid_max_radius_factor` , default `3.0` , matching the pilot script's own default) rather than pilot-specific. Tested by deliberately inflating a centroid to 1000x its layer's scale mid-training and confirming one real `train_step` brings it back within bounds (`test_train_step_clips_centroid_norms_after_optimizer_step`) — the reproduction test in `test_centroids.py` already proves `clip_norm` itself works; this one proves `Trainer` actually calls it.

5. Training Metrics Reference

Every metric logged during a real training run (`Trainer` , `pilot_finetune.py` , or both), what it's actually computing, and how to read it. All are also covered by `tests/training/test_losses.py` and `tests/training/test_trainer.py` .

5.1 `task_loss`

Standard causal-LM cross-entropy (`training/losses.py::total_loss`), computed over every non-padded position (`IGNORE_INDEX = -100` masking matches `collate_fn` 's shifted-label convention). This is the metric that actually matters for model quality — `load_loss` / `sep_loss` below exist only to shape *how* routing behaves, not to directly improve prediction. Reported in nats (natural-log cross-entropy), not bits or perplexity; `evaluate.py` / `Trainer.evaluate()` separately reports validation perplexity (`exp(nll)` , capped at `exp(20)` to avoid `inf` on a garbage model).

5.2 `load_loss` — `routing/load_balance.py::coefficient_of_variation_loss`

The squared coefficient of variation ($(\text{std}/\text{mean})^2$) of each expert's average share of the *dense* (untempered, pre-top-k) softmax gate mass across a batch. Concretely: `load = softmax(router_logits).reshape(-1, n_experts).mean(dim=0)` , then `load_loss = (load.std() / load.mean())2` .

- **0.0** — perfectly balanced: every expert receives, on average, exactly `1/n_experts` of the gate mass.
- **n_experts - 1** (its exact maximum — `7.0` at the project's default `n_experts=8`) — total collapse: all gate mass concentrated on a single expert. This value has been observed exactly, repeatedly, in real training runs (§4.5) — it's not a theoretical edge case.
- Computed identically across **all four** router variants (`DiffusionMoELayer` , `CosineMoELayer` , `SwitchMoELayer` , `RandomMoELayer`), since every variant's aux dict carries dense `router_logits` — directly comparable between them.
- **Per-layer breakdown**: `total_loss` also returns `load_loss/layer_{idx}` for every MoE layer present (added specifically because the aggregate mean hides exactly what's needed to answer "did *this* layer's router collapse, independent of what every other layer did" — see findings report §3.6/the "what happens if a layer keeps forcing a single expert" discussion). The top-level `load_loss` is the mean of these per-layer values.

5.3 `sep_loss` — `routing/separation.py::centroid_separation_loss`

Negative mean pairwise Euclidean distance between expert centroids, in `landmark_scale` -normalized diffusion coordinates (so the value is comparable across layers and training progress rather than growing with raw centroid distance — see `landmark_scale` 's own docstring for why the guarding epsilon is `1e-30` , not the more conventional `1e-8`). Minimizing this loss pushes centroids apart, encouraging distinct experts to specialize on distinct regions of diffusion space.

- **More negative = centroids more spread apart**. There is **no lower bound** on this loss by design (`test_more_spread_centroids_give_more_negative_loss` confirms this is intentional, not a bug) — nothing in the loss itself stops centroids drifting arbitrarily far outside the data's real coordinate range, which is exactly what happened in the first real pilot run (`sep_loss` grew from `-3` to `-19,185` over 300 steps) before `ExpertCentroids.clip_norm()` was added as an external safeguard (§4.4).
- **Only computed for layers whose aux dict carries both centroids and Psi_landmarks** — currently `DiffusionMoELayer` only (`CosineMoELayer` 's centroids live in raw `d_model` space, not diffusion coordinates, so this loss doesn't apply to them). **0.0** if no layer has both keys (e.g. a fully dense model, or a run using only baseline router variants).

- **Per-layer breakdown:** same pattern as `load_loss` — `total_loss` returns `sep_loss/layer_{idx}` per applicable layer; the top-level `sep_loss` is their mean.

5.4 `centroid_norm_mean` , `per_expert_load` , `argmax_expert` — `pilot_finetune.py` only

Not part of `total_loss` ; logged directly by the pilot script's own training loop (not currently mirrored in `Trainer` , since they were added for interactive debugging of one layer at a time, not general production logging):

- **`centroid_norm_mean`** — mean L2 norm of the layer's `n_experts` centroids, post- `clip_norm_` . A sanity check that the norm-capping safeguard is actually holding (should stay near, not above, `centroid_max_radius_factor × landmark_scale`).
- **`per_expert_load`** — the same dense softmax mean as `load_loss` computes, but returned as the full `(n_experts,)` vector rather than reduced to one scalar. This is what actually let the seed-dependence finding happen (§3.5): `load_loss` alone would show "collapsed" in both the seed-42 and seed-123 pilot runs identically, but only the full vector reveals collapse landed on a *different* expert each time.
- **`argmax_expert`** — `per_expert_load.argmax()` , the single most-favored expert that step. The cheap summary of `per_expert_load` used for the "which expert wins 83% of steps" style analysis throughout §3.5.

5.5 `loss` — the actual optimization target

`loss = task_loss + mu * load_loss + nu * sep_loss` (`total_loss` 's return value with `.backward()` called on it). `mu / nu` default to `2.0 / 0.05` (`routing.mu_load / routing.nu_sep`). Everything else in this section is diagnostic/logging output, detached from the graph before being reported — only this composite scalar actually drives gradients.

`mu_load` was `0.01` until `phase2_training_report.md` §6.2: measured directly (not guessed) on a freshly, correctly-initialized model, `task_loss` 's raw gradient pull on the shared `ExpertCentroids` parameter is only ~1.86x `load_loss` 's raw pull — `mu_load=0.01` shrank that to an effective ~186x in `task_loss` 's favor, leaving `load_loss` almost no real ability to resist router collapse. `nu_sep` has not been measured the same way yet — see that report's appendix.

6. All-Layers Training Run: Architecture and Infrastructure Additions

Durable reference material for mechanisms added while training every layer of the project's own from-scratch architecture as `DiffusionMoELayer` — see `phase2_training_report.md` for the narrative (bugs found, evidence, run-by-run results). This section covers what the mechanisms *are*; that report covers *why* and *what happened*.

6.1 `DiffusionMoELayer.shared_expert`

Every token, every step, passes through one additional `ExpertFFN` (`shared_expert`) that is applied unconditionally — no gate value, no centroid, no participation in top-k selection, entirely outside the diffusion router. Architecturally identical to a routed expert (same `hidden_dim = ffn_dim // n_experts * overlap_factor` , same `overlap_factor`); the only difference is that it always fires. Adds real active compute per token (`top_k + 1` experts instead of `top_k`), not a reallocation of existing capacity. Precedented in production MoE architectures (DeepSeekMoE's "shared expert isolation," similarly Qwen2-MoE) as a way to give the network *some* routing-independent gradient signal, so common/generic computation isn't entirely contingent on that step's routing decisions.

6.2 Nystrom kernel bandwidth: live refresh between refits

`NystromDiffusionMap.transform(Z, _refresh_eps=True)` (the default for a standalone call — i.e. every non-refit training step) re-derives `eps_` (the kernel bandwidth) from the current batch's actual distance to the frozen landmarks, rather than reusing the value fixed at the last `fit()` . Reuses the distance matrix already computed for the Nyström kernel itself, so the marginal cost is one more `n_landmarks × n_landmarks` eigensolve (`_fit_landmark_spectrum`), not another k-means clustering pass. `fit_transform()` 's own internal `transform()` call passes `_refresh_eps=False` , since immediately after a fresh `fit()` there is no staleness to correct. An explicit `eps` override (used by `geometry/multiscale.py` 's dyadic sweep) disables the refresh entirely,

preserved via the same `self.eps is None` gate `fit()` already used to decide whether to apply the median heuristic.

Why this exists: a kernel bandwidth fit once and frozen between refits describes a snapshot of the embedding distribution that grows more stale the longer training runs — see `phase2_training_report.md §2` for the NaN-divergence incident this was found to cause.

6.3 Checkpoint retention

`Trainer._prune_old_checkpoints()`, called after every numbered checkpoint save: deletes all but the most recent `training.keep_last_n_checkpoints` (default 2) numbered checkpoints (`step_N.pt`). `best.pt` is a separate file, always kept regardless. `keep_last_n_checkpoints <= 0` disables pruning (unbounded retention, the old default behavior) as an opt-in.

6.4 `scripts/expert_attribution.py` : Hydra CLI overrides

`load_config(config_name, overrides)` now accepts leftover `key=value` arguments (via `argparse.parse_known_args`), the same convention `scripts/train.py` already supports — needed because `build_dataloaders(cfg)` otherwise always falls back to `base_config.yaml`'s own data defaults (`the_pile`, Hub streaming, `max_seq_len=2048`), which will not match whatever data config a given checkpoint was actually trained against. Example: `python scripts/expert_attribution.py --checkpoint checkpoints/step_300.pt data.dataset=wikitext data.local_data_files=[...] data.max_seq_len=512`.

6.5 Model initialization

`DiffusionMoETransformer.__init__` now explicitly initializes two things that previously relied on PyTorch's untouched defaults:

- **token_embedding.weight** : `Normal(0, 0.02)` (was `nn.Embedding`'s default `Normal(0, 1)`). Matters more than usual here because `tie_embeddings=True` by default makes this same matrix double as the LM-head unembedding projection.
- **Every residual-branch output projection** (attention's `out_proj`, every FFN's `down_proj` — dense `TransformerBlock`, routed `ExpertFFN`, and `shared_expert` alike): scaled by `1/sqrt(2 × n_layers)` after construction, matched by the Linear's own attribute name rather than by module class (so it applies uniformly regardless of which kind of block it's nested in). Standard GPT-2/nanoGPT convention, previously entirely absent from this model.

Sanity check available as a reusable pattern, not just a one-off: cross-entropy of a fresh random-init model against random labels should land near `ln(vocab_size)` (this project's default: `ln(32000) ≈ 10.37`). A value dramatically higher (or, differently, exactly the cap value seen repeatedly at eval time) is a strong, cheap signal of an initialization-scale problem, checkable before spending any real training compute — see `phase2_training_report.md §5` for the incident this check would have caught immediately.

6.6 Compute-matching a routed layer against dense: the active-FFN-FLOPs formula

Any dense-vs-MoE comparison in this codebase needs both sides running the same active FLOPs/token to mean anything — parameter count alone is misleading, since most of a routed layer's parameters are inert for any given token. `ExpertFFN`'s hidden width is `ffn_dim // n_experts × overlap_factor` (not `ffn_dim` itself — that's the *total* capacity split across `n_experts`), and each token activates `top_k` routed experts plus the always-on `shared_expert` (§6.1) when enabled. So:

```
active_ffn_width_per_token = (top_k + int(use_shared_expert)) × (ffn_dim // n_experts × overlap_factor)
```

A compute-matched dense comparison sets the plain `TransformerBlock`'s `ffn_dim` (`model.ffn_dim`) to this value directly, since dense's `FeedForward` uses its configured `ffn_dim` as-is with no division. At this project's `base_config.yaml` defaults (`n_experts=8`, `top_k=2`, `ffn_dim=4096`, `overlap_factor=1.0`, `use_shared_expert=true`): `active_ffn_width = 3 × 512 = 1536`, dramatically less than the `ffn_dim=4096`

a naive dense-baseline config would use. `phase2_training_report.md` §8.9 found the uncorrected 2.7x compute gap had been silently favoring dense in every prior comparison in that report (§6.4, §6.5, §8.6, §8.8) — worth checking explicitly, not assumed, in any future dense-vs-routed comparison.

6.7 Expert-separation / duplication check: token attribution + weight cosine similarity

Neither load-balance loss in this codebase (`coefficient_of_variation_loss` , `switch_load_balance_loss`) penalizes anything beyond marginal per-expert dispatch *frequency* — a router that consistently pairs experts into redundant duplicate groups would score as perfectly "balanced" under either, since both are satisfied by symmetric duplication just as well as by genuine functional diversity. Two complementary checks, run against a trained checkpoint (`phase2_training_report.md` §8.8 has the worked example, Switch layer 0, `n_experts=8`):

- **Token-identity overlap:** `scripts/expert_attribution.py --checkpoint <path> --layers <idx>` (needs the checkpoint's actual data config passed as overrides — see §6.4) produces each expert's top-N most-routed tokens. Pairwise Jaccard overlap of these top-token sets across all expert pairs is a cheap proxy for functional overlap — genuinely redundant experts would show overlap approaching 1.0 for at least one pair; §8.8's result averaged 0.08 (range 0.00-0.21).
- **Weight-space similarity:** flatten each expert's `down_proj.weight` (or any single projection, held consistent across experts) and compute pairwise cosine similarity; compare against a random-initialization baseline of the same shape as the null hypothesis (high-dimensional random vectors are already near-orthogonal, so "near zero" alone doesn't prove non-redundancy — the informative comparison is trained-vs-random, not trained-vs-zero). §8.8's trained similarities (mean -0.0005, max 0.007) were statistically indistinguishable from the random baseline (mean 0.0002, max 0.003).

Neither check is definitive on its own (token-overlap says nothing about internal weight structure; weight-cosine in a ~500K-dim space is a weak test for redundancy expressed via internal hidden-unit permutation rather than raw parameter alignment) — a functional-output-correlation check (comparing experts' actual outputs on a shared batch of real activations, which is invariant to the internal-permutation blind spot) would be the more rigorous version if either check's result is ambiguous or a future run's finding needs to be trusted more strongly.